

RAJIV GANDHI COLLEGE OF ENGINEERING AND TECHNOLOGY
KIRUMAMPAKKAM, PUDUCHERRY-607403.
(Approved by AICTE and Affiliated to Pondicherry University)
(ISO 9001:2015)



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

COURSE FILE

Staff Name : Mrs.D.SUGANYA/Mrs.B.VIDHYA

Designation : AP(ST-II)/AP(ST-I)

Academic Year : 2023-2024 (EVEN SEM)

Course : B. TECH (CSE)

Lab Subject & Code : EMBEDDED SYSTEMS LAB(CS P62)

Subject Name EMBEDDED SYSTEMS LABORATORY
Subject Code: CS P62

Program Outcomes (POs)

The **Embedded Systems Laboratory** course focuses on practical experimentation and hands-on development of embedded systems applications. The experiments are designed to provide exposure to ARM-based processors and Embedded Development Environments (IDE), along with Real-Time Operating Systems (RTOS) for task management and peripheral interfacing.

Course Outcomes (COs)

1. **CO1:** Demonstrate proficiency in writing assembly programs for basic arithmetic operations, system calls, and interrupts on ARM processors.
 2. **CO2:** Develop assembly programs to configure and control general-purpose I/O ports and interface external peripherals with embedded systems.
 3. **CO3:** Implement time delays, interrupt handling, and basic peripheral interfaces such as LEDs, switches, and buzzers in embedded systems.
 4. **CO4:** Interface I2C devices (such as Serial EEPROM) and demonstrate serial communication in embedded systems.
 5. **CO5:** Apply RTOS concepts to create and manage tasks, interrupts, and inter-task communication on ARM processors using an embedded IDE.
-

Alignment of COs with Pos and PSOs:

The Program Outcomes (POs), Program Specific Outcomes (PSOs) and their corresponding achievements through Course Outcomes (COs) are detailed below.

Program Outcomes (POs) Based on NBA Guidelines:

1. **Engineering Knowledge**
2. **Problem Analysis**
3. **Design/Development of Solutions**
4. **Conduct Investigations of Complex Problems**
5. **Modern Tool Usage**
6. **The Engineer and Society**

7. **Environment and Sustainability**
 8. **Ethics**
 9. **Individual and Team Work**
 10. **Communication**
 11. **Project Management and Finance**
 12. **Lifelong Learning**
-

Program Specific Outcomes (PSOs):

- **PSO1:** Apply foundational principles of computer science, engineering, and mathematics to develop computational systems and solve complex problems.
 - **PSO2:** Design and implement software solutions for diverse real-world problems using modern technologies and programming practices.
 - **PSO3:** Demonstrate expertise in specialized areas such as artificial intelligence, data science, cybersecurity, and software development to contribute to research and industry advancements.
-

Explanation of How POs Are Achieved

1. **CO1:** Demonstrate proficiency in writing assembly programs for basic arithmetic operations, system calls, and interrupts on ARM processors.
 - **PO1 (Engineering Knowledge):** Students apply fundamental concepts of embedded systems and microprocessor operations.
 - **PO3 (Design/Development of Solutions):** Writing assembly programs to perform basic operations and control system resources demonstrates design and implementation capabilities.
 - **PO5 (Modern Tool Usage):** Students use embedded development tools and IDEs for assembly programming on ARM processors.
 - **PSO1:** Students gain hands-on experience in developing embedded applications on ARM processors.
2. **CO2:** Develop assembly programs to configure and control general-purpose I/O ports and interface external peripherals with embedded systems.
 - **PO3 (Design/Development of Solutions):** The configuration and control of GPIO ports require the application of design principles to interface peripherals effectively.
 - **PO5 (Modern Tool Usage):** Students utilize embedded IDEs to configure and control peripherals through assembly programming.

- **PSO1:** This CO directly relates to interfacing embedded systems with external devices, enhancing practical skills in embedded development.
 - 3. **CO3:** Implement time delays, interrupt handling, and basic peripheral interfaces such as LEDs, switches, and buzzers in embedded systems.
 - **PO3 (Design/Development of Solutions):** Designing interrupt service routines (ISRs) and implementing peripheral interfaces are critical skills in embedded systems development.
 - **PO5 (Modern Tool Usage):** Students use IDEs to simulate and debug interrupt handling and peripheral interfaces in embedded systems.
 - **PSO1:** The hands-on work with peripherals (LEDs, switches, buzzers) and interrupts supports the development of embedded applications with real-time requirements.
 - 4. **CO4:** Interface I2C devices (such as Serial EEPROM) and demonstrate serial communication in embedded systems.
 - **PO3 (Design/Development of Solutions):** Students design and implement I2C communication and serial communication protocols to interface with external devices.
 - **PO5 (Modern Tool Usage):** Using IDEs and embedded tools, students develop I2C and serial communication applications for data exchange.
 - **PSO1:** This CO supports the development of embedded systems with advanced peripheral communication techniques (I2C and serial communication).
 - 5. **CO5:** Apply RTOS concepts to create and manage tasks, interrupts, and inter-task communication on ARM processors using an embedded IDE.
 - **PO1 (Engineering Knowledge):** Students apply RTOS concepts and task scheduling mechanisms to manage embedded system processes.
 - **PO3 (Design/Development of Solutions):** Developing tasks, managing interrupts, and inter-task communication in a multi-tasking environment demonstrates the application of RTOS principles.
 - **PO5 (Modern Tool Usage):** Using RTOS tools in an IDE environment, students implement tasks and manage system resources efficiently.
 - **PSO2:** Students apply RTOS concepts to manage and optimize embedded system processes, supporting real-time application development.
-

CO-PO-PSO Matrix

Course Outcomes (COs)	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12	PSO1	PSO2	PSO3
CO1	3	2	3	1	3	2	1	0	0	1	0	0	3	2	0
CO2	3	2	3	1	3	2	1	0	0	1	0	0	3	2	1
CO3	3	2	3	2	3	3	1	1	0	1	0	1	3	3	2
CO4	3	2	3	2	3	3	1	1	0	2	1	1	3	3	3
CO5	3	2	3	2	3	3	1	1	0	2	1	1	3	3	3

Explanation of the CO-PO-PSO Matrix

- **PO1 (Engineering Knowledge):** COs such as CO1, CO2, CO3, CO4, and CO5 require students to apply their knowledge of embedded systems, assembly programming, and microcontroller architecture to design and implement solutions.
- **PO3 (Design/Development of Solutions):** All COs require the students to design and develop embedded system applications, particularly in assembly programming, interrupt handling, and RTOS management.
- **PO5 (Modern Tool Usage):** Students use embedded IDE tools and modern ARM-based development environments to write, test, and debug programs.
- **PSO1:** All COs contribute to the development of embedded systems applications using ARM processors and peripheral interfaces.
- **PSO2:** CO5 primarily supports the application of RTOS concepts to manage tasks and interrupts, which is essential for real-time embedded applications.
- **PSO3:** COs contribute to peripheral interfacing, serial communication, and inter-task communication, providing a well-rounded embedded systems development experience.

RAJIV GANDHI COLLEGE OF ENGINEERING AND TECHNOLOGY

KIRUMAMPAKKAM, PUDUCHERRY-607403.

(Approved by AICTE and Affiliated to Pondicherry University) (ISO 9001:2015)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

LABORATORY MANUAL



Regulation – 2013-2014

CS P62 – EMBEDDED SYSTEMS LABORATORY

III YEAR VI SEMESTER

Subject Code	Subject Name	Lectures (Periods)	Tutorials (Periods)	Practical (Periods)
CS P62	EMBEDDED SYSTEMS LABORATORY	-	-	3

LIST OF EXPERIMENTS

The following programs are to be implemented on ARM based Processors/Equivalent.

1. Simple Assembly Program for Addition, Subtraction, Multiplication and Division
2. Simple Assembly Program for System Calls and Interrupts, Loops and Branches
3. Write an Assembly programs to configure and control General Purpose Input/Output (GPIO) port pins.
4. Write an Assembly programs to read digital values from external peripherals and execute them with the Target board.
5. Program to demonstrate Time delay program using built in Timer / Counter feature on IDE environment
6. Program to demonstrate a simple interrupt handler and setting up a timer
7. Program to Interface 8 Bit LED and Switch Interface
8. Program to implement Buzzer Interface on IDE environment Program to Displaying a message in a 2 line x 16 Characters LCD display and verify the result in debug terminal.
9. Program to demonstrate I²C Interface on IDE environment
10. Program to demonstrate I²C Interface – Serial EEPROM
11. Demonstration of Serial communication. Transmission from Kit and reception from PC using
12. Serial Port on IDE environment use debug terminal to trace the program.

Write the following programs to understand the use of RTOS with ARM Processor on IDE Environment using ARM Tool chain and Library:

1. Write an application that creates a task which is scheduled when a button is pressed, which illustrates the use of an event set between an ISR and a task
2. Write an application that Demonstrates the interruptible ISRs(Requires timer to have higher priority than external interrupt button)
3. Write an application that creates a two task to Blinking two different LEDs at different timings
4. Write an application that creates a two task displaying two different messages in LCD display in two lines.
5. Sending messages to mailbox by one task and reading the message from mailbox by another task.
6. Sending message to PC through serial port by three different tasks on priority Basis.
7. Basic Audio Processing on IDE environment.

LIST OF EXPERIMENTS

Expt. No.	Title of the Experiment
1	Study of ARM Processor
2	Simple Assembly Program for Addition
3	Simple Assembly Program for Subtraction
4	Simple Assembly Program for Multiplication
5	Simple Assembly Program for Division
6	Simple Assembly Program for System Calls and Interrupts, Loops and Branches
7	Program to demonstrate Time delay program using built in Timer
8	Program to Interface 8 Bit LED and Switch Interface
9	Program to implement Buzzer Interface
10	Program to demonstrate I2C Interface on IDE environment
11	Program to demonstrate I2C Interface – Serial EEPROM
12	To creates a two task to Blinking two different LEDs at different timings
13	To creates a two-task displaying two different messages in LCD display in two lines.

Study of ARM Processor

Aim

To learn about the evolution, core features, general characteristics and applications of ARM processors.

Pre Lab Questions

1. What is an embedded system?
2. Mention the difference between microprocessor and microcontroller.
3. Enumerate the terms object oriented and object based language
4. Define Pipelining.
5. List the basic units of Microprocessor

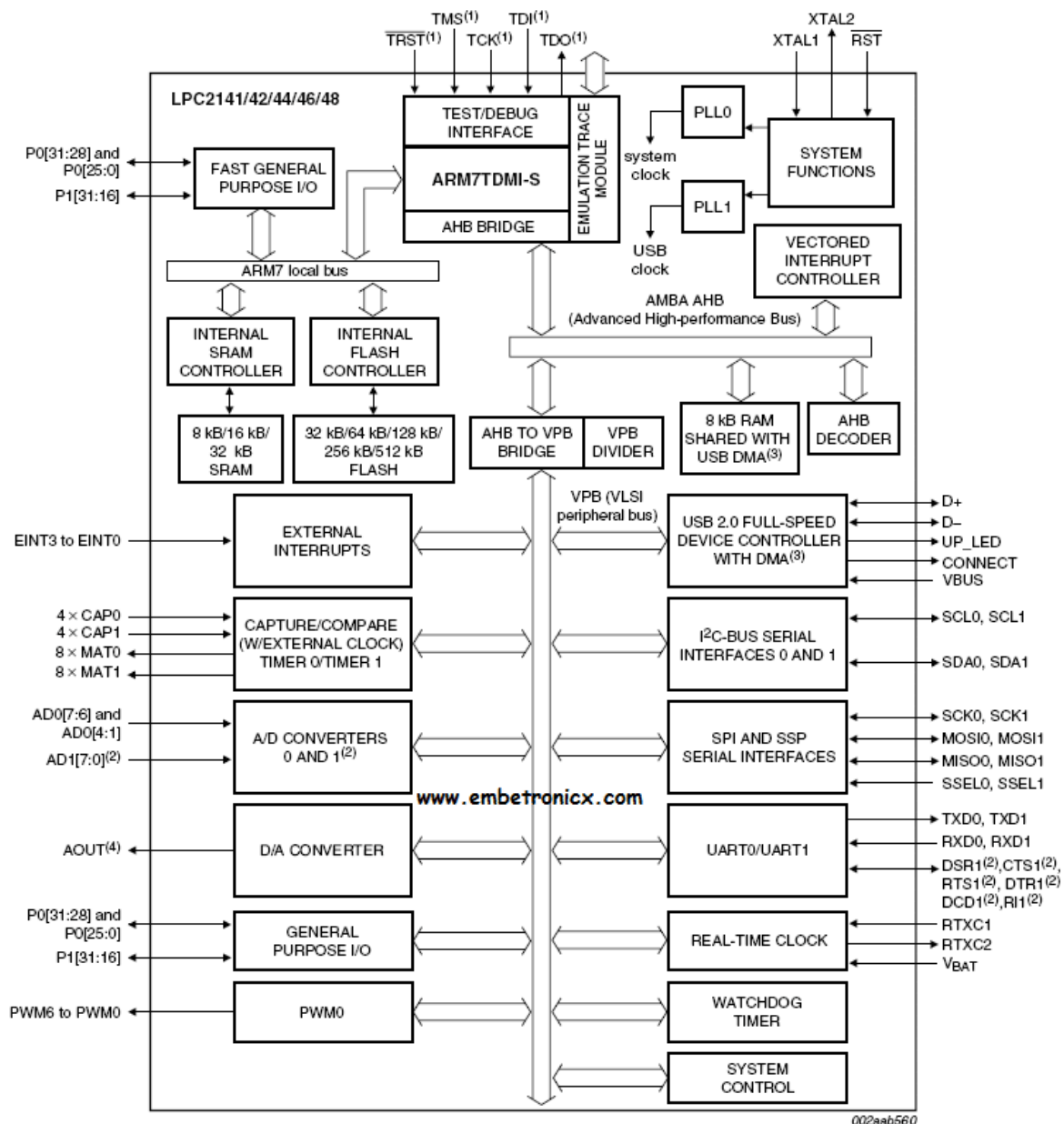
Theory

The LPC2148 microcontrollers are based on a 32/16 bit ARM7TDMI-S CPU with real- time emulation and embedded trace support, that combines the microcontroller with embedded high speed flash memory ranging from 32 kB to 512 kB. A 128-bit wide memory interface and unique accelerator architecture enable 32-bit code execution at the maximum clock rate. For critical code size applications, the alternative 16- bit Thumb mode reduces code by more than 30 % with minimal performance penalty.

Due to their tiny size and low power consumption, LPC2148 are ideal for applications where miniaturization is a key requirement, such as access control and point- of-sale. A blend of serial communications interfaces ranging from a USB 2.0 Full Speed device, multiple UARTS, SPI, SSP to I2C s and on-chip SRAM of 8 kb up to 40 kb, make these devices very well suited for communication gateways and protocol converters, soft modems, voice recognition and low end imaging, providing both large buffer size and high processing power. Various 32-bit timers, single or dual 10-bit ADC(s), 10-bit DAC, PWM channels and 45 fast GPIO lines with up to nine edge or level sensitive external interrupt pins make these microcontrollers

particularly suitable for industrial control and medical systems.

ARM7 LPC 2148 ARCHITECTURE



(1) Pins shared with GPIO.

(2) LPC2144/46/48 only.

(3) USB DMA controller with 8 kB of RAM accessible as general purpose RAM and/or DMA is available in LPC2146/48 only.

(4) LPC2142/44/46/48 only.

LPC2148 specification

Features:

16/32-bit ARM7TDMI-S microcontroller in a tiny LQFP64 package.

8 to 40 KB of on-chip static RAM and 32 to 512kB of on-chip flash Program memory.

128 bit wide interface/accelerator enables high speed 60 MHz operation.

In-System/In-Application Programming (ISP/IAP) via on-chip boot-loader software. Single flash sector or full chip erase in 400ms and programming of 256 bytes in 1 ms.

Embedded ICE RT and Embedded Trace interfaces offer real-time debugging with the on-chip Real Monitor software and high speed tracing of instruction execution.

USB 2.0 Full Speed compliant Device Controller with 2 kb of endpoint RAM. In addition, the LPC2146/8 provide 8 kb of on-chip RAM accessible to USB by DMA.

One or two (LPC2141/2 vs. LPC2144/6/8) 10-bit A/D converters provide a

- total of 6/14 analog inputs, with conversion times as low as 2.44µs per channel.

Single 10-bit D/A converter provide variable analog output.

Two 32-bit timers/external event counters (with four capture and four compare channels each), PWM unit (six outputs) and watchdog.

Low power real-time clock with independent power and dedicated 32kHz Clock input.

Multiple serial interfaces including two UARTs (16C550), two Fast I2C -bus (400 kbit/s), SPI and SSP with buffering and variable data length capabilities.

Vectored interrupt controller with configurable priorities and vector addresses.

Up to 45 of 5 V tolerant fast general purpose I/O pins in a tinyLQFP64 package.

Up to nine edge or level sensitive external interrupt pins available.

60 MHz maximum CPU clock available from programmable on-chip PLL with settling time of 100 μ s.

On-chip integrated oscillator operates with an external crystal in range from 1MHz to 30 MHz and with an external oscillator up to 50MHz.

Power saving modes include Idle and Power-down.

Individual enable/disable of peripheral functions as well as peripheral clock scaling for additional power optimization.

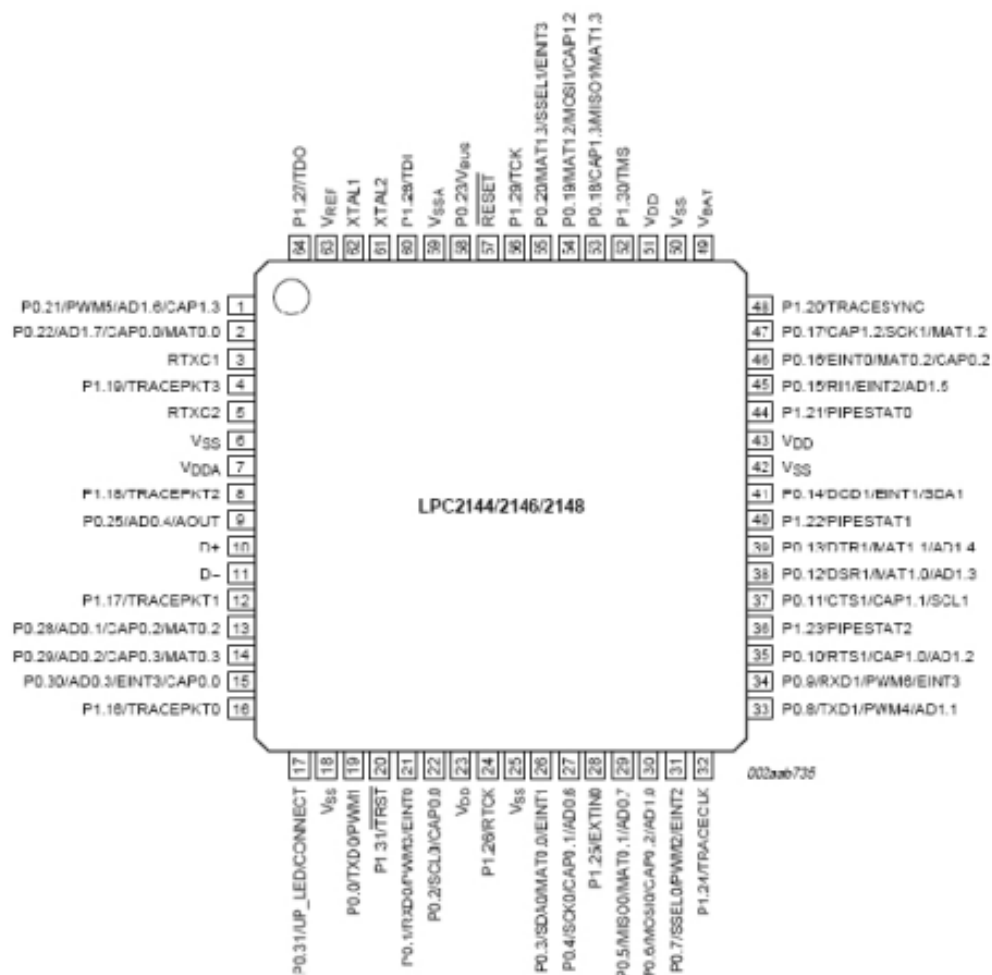
Processor wake-up from Power-down mode via external interrupt, USB,

Brown-Out Detect (BOD) or Real-Time Clock(RTC).

Single power supply chip with Power-On Reset (POR) and BOD circuits:

- CPU operating voltage range of 3.0 V to 3.6 V ($3.3\text{ V} \pm 10\%$) with 5V tolerant I/O pads.

Pin Configuration:

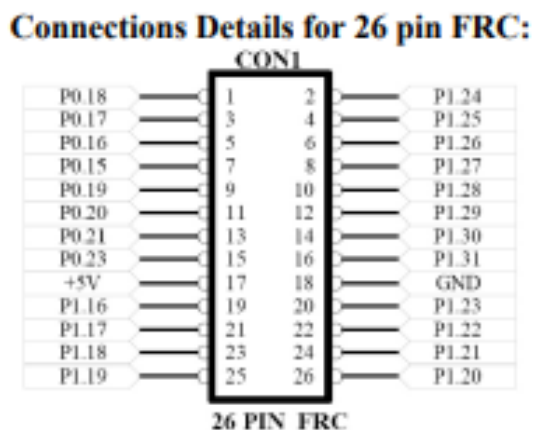


For further studies about LPC2148 specification refer NXP's website to download LPC2148 user manual.

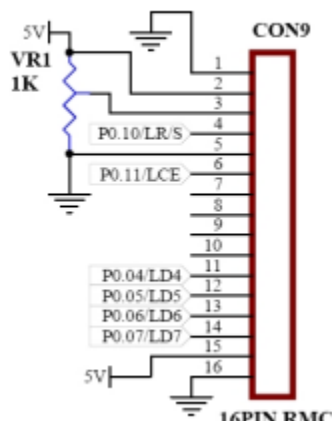
APPENDIX-II: CONNECTORS & CONNECTION DETAILS

Type	Label	Description
26 Pin FRC	CON1	For GPIO
20 Pin FRC	CON2	For JTAG
DB9	CON3	UART0
DB9	CON4	UART1
3 Pin RMC	CON5	PWM output
6 Pin RMC	CON6	ADC input
3 Pin RMC	CON7	DAC output from OPAMP
3 Pin RMC	CON8	Temperature sensor input
16 Pin RMC	CON9	LCD Display
Power jack	C10	Power input
PS2	CON11	Keyboard interface

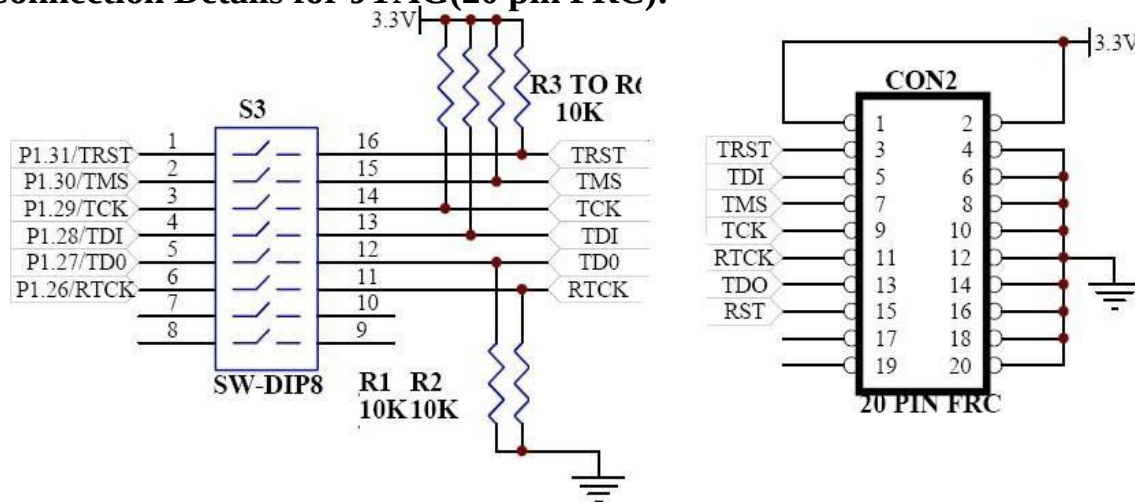
Connections Details for 26 pin FRC:



Connection Details for LCD(16 pin RMC):



Connection Details for JTAG(20 pin FRC):



Post Lab Questions

1. What is the difference between embedded systems and the system in which rtos is running?
2. Discuss about semaphore.
3. What are the instructions used to access the memory in ARM?
4. Mention the characteristics of RISK Instruction.
5. Define Interrupt.

Result

The evolution, core features, general characteristics and the applications of ARM processors has been studied and is evaluated.

Simple Assembly Program for Addition

Aim

To write an ARM assembly language program to perform the addition of two hexadecimal numbers and store the result in a register. The program demonstrates basic loading of data, arithmetic operations, and program control flow

.

Procedure:

1. Initialize the Program:

- Define the program's starting point using the label `__main`.
- Use the `AREA` directive to specify the memory region for the code segment.

2. Data Loading:

- Use the `LDR` instruction to load two hexadecimal values (e.g., `0x00011111` and `0x54711212`) into registers `R0` and `R1`.

3. Perform Addition:

- Use the `ADD` instruction to compute the sum of the values in `R0` and `R1`.
- Store the result in register `R2`.

4. Control Flow:

- After the addition, introduce an infinite loop to halt further execution using the `B` instruction with the label `stop`.

5. End the Program:

- Use the `END` directive to indicate the program's termination.

6. Export Symbols:

- Use `EXPORT` directives to make symbols accessible to the linker, such as `__main` and system-specific symbols like `__use_two_region_memory`.

Apparatus & Software Required

1. LPC2148 Development board.

2. KeilµVision 5 software.
3. Flash Magic.
4. USB cable.
5. CRO.

PROGRAM

```

GLOBAL __main
                AREA main, CODE, READONLY
                EXPORT __main
                EXPORT __use_two_region_memory
__use_two_region_memory EQU 0
                BX    LR
                ENTRY

;Main loop begins

__main
                LDR R0,=0X00011111 ;load first input
                LDR R1,=0X54711212 ;load second input
                ADD r2,r0,r1

stop           B stop

                end

```

Result of the Program:

The addition of the two hexadecimal numbers:

- First Input: 0x00011111
- Second Input: 0x54711212

Result in Register R2: 0x54722323 (in hexadecimal) or 1416766243 (in decimal).

This result will be stored in register R2 after the execution of the program

Simple Assembly Program for Subtraction

Aim

To write an ARM assembly language program to perform the subtraction of two hexadecimal numbers and store the result in a register. This program demonstrates basic data loading, arithmetic operations, and control flow in ARM assembly.

Procedure

1. Initialize the Program:

- Use the AREA directive to define the code region and mark it as read-only.
- Set the entry point using the label __main.

2. Load Data into Registers:

- Load the first hexadecimal number (0x00011111) into register R0 using the LDR instruction.
- Load the second hexadecimal number (0x54711212) into register R1.

3. Perform Subtraction:

- Use the SUB instruction to subtract the value in R1 from the value in R0.
- Store the result in register R2.

4. Control Flow:

- After the subtraction, add an infinite loop using the B instruction with the label stop to prevent further execution.

5. Export Symbols:

- Use EXPORT directives to make __main and other necessary symbols accessible to the linker.

6. End the Program:

- Use the END directive to indicate the termination of the program.

Program:

```

GLOBAL __main
        AREA main, CODE, READONLY
        EXPORT __main
        EXPORT __use_two_region_memory
__use_two_region_memory EQU 0
        BX    LR
        ENTRY

;Main loop begins

__main
        LDR R0,=0X00011111 ;load first input
        LDR R1,=0X54711212 ;load second input
        SUB r2,r0,r1

stop      B stop

        end

```

Expected Output

- The subtraction of 0x54711212 from 0x00011111 will result in a negative value, stored in **register R2**.
- The program halts in an infinite loop (stop) after performing the subtraction.

Simple Assembly Program for Multiplication

Aim

To write an ARM assembly language program to perform the multiplication of two hexadecimal numbers and store the result in a register. This program demonstrates the use of the MUL instruction for performing arithmetic operations in ARM assembly.

Procedure

1. Initialize the Program:

- Use the AREA directive to specify the memory region for the code.
- Define the starting point of the program using the label __main.

2. Load Data into Registers:

- Load the first hexadecimal number (0x00011111) into register R0 using the LDR instruction.
- Load the second hexadecimal number (0x54711212) into register R1.

3. Perform Multiplication:

- Use the MUL instruction to multiply the values in R0 and R1.
- Store the multiplication result in register R2.

4. Control Flow:

- Introduce an infinite loop using the B instruction with the label stop to halt the program after the multiplication is performed.

5. Export Symbols:

- Use EXPORT directives to make the symbols __main and __use_two_region_memory accessible to the linker.

6. End the Program:

- Use the END directive to indicate the termination of the program.

Program

```

GLOBAL __main

        AREA main, CODE, READONLY
        EXPORT __main
        EXPORT __use_two_region_memory
__use_two_region_memory EQU 0

        BX    LR
        ENTRY

;Main loop begins

__main

        LDR R0,=0X00011111 ;load first input
        LDR R1,=0X54711212 ;load second input
        MUL  r2,r0,r1

stop      B stop

        end

```

Expected Output

- The multiplication of 0x00011111 and 0x54711212 will be stored in **register R2**.
- The program halts in an infinite loop (stop) after computing the multiplication.

This program helps understand the use of the MUL instruction and the handling of basic arithmetic operations in ARM assembly programming.

Simple Assembly Program for Division

Aim

To write an ARM assembly language program to perform addition, subtraction, and multiplication on two hexadecimal numbers. The results are stored in separate registers, demonstrating the use of basic arithmetic operations and flags in ARM assembly.

Procedure

1. Initialize the Program:

- Use the AREA directive to specify the memory region for the code.
- Define the starting point of the program with the label __main.

2. Load Data into Registers:

- Load the first hexadecimal number (0x00000006) into register R0 using the LDR instruction.
- Load the second hexadecimal number (0x00000004) into register R1.

3. Perform Arithmetic Operations:

- **Addition:**
 - Use the ADD instruction to add the values in R0 and R1.
 - Store the result in register R2.
- **Subtraction:**
 - Use the SUBS instruction to subtract the value in R1 from R0.

- Store the result in register R3. The SUBS instruction updates flags based on the result.
- **Multiplication:**
 - Use the MUL instruction to multiply the values in R0 and R1.
 - Store the result in register R4.

4. Control Flow:

- After performing the operations, introduce an infinite loop using the B instruction with the label stop to halt further execution.

5. Export Symbols:

- Use the EXPORT directives to make __main and __use_two_region_memory accessible to the linker.

6. End the Program:

- Use the END directive to indicate the termination of the program.

Source program

```
GLOBAL __main
        AREA main, CODE, READONLY
        EXPORT __main
        EXPORT __use_two_region_memory
__use_two_region_memory EQU 0
        BX    LR
        ENTRY
```

;Main loop begins

```
__main
```

```
LDR R0,=0x00000006 ;load first input
LDR R1,=0x00000004 ;load second input
ADD r2,R0,R1
SUBS R3,R0,R1
MUL R4,R0,R1
```

stop

B stop

end

Expected Output

- **Addition Result** (R2): 0x0000000A (decimal: 10).
- **Subtraction Result** (R3): 0x00000002 (decimal: 2).
- **Multiplication Result** (R4): 0x00000018 (decimal: 24).
- The program halts in an infinite loop (stop) after performing the operations.

Aim

To write a simple ARM assembly program demonstrating system calls, interrupts, loops, and branches.

Procedure

1. Initialize Registers:

- Load the loop counter into R0 and the increment value into R1.
- Set R2 to 0 to accumulate the result.

2. Implement Loop with Branching:

- Add the value in R1 to R2.
- Decrement the loop counter (R0) and check the condition using the SUBS instruction.
- Use BNE (Branch if Not Equal) to repeat the loop until R0 becomes zero.

3. System Call Simulation:

- Use the software interrupt (SWI) to simulate a system call (e.g., printing the result).
- MOV R7, #4 is a placeholder for invoking a print operation (Linux syscall convention).

4. Halt the Program:

Use an infinite loop (B stop) to prevent further execution

Program

```

asm
Copy code
    AREA main, CODE, READONLY
    EXPORT __main
    EXPORT __use_two_region_memory
    __use_two_region_memory EQU 0
    ENTRY

; Main program begins
__main
    LDR R0, =10      ; Load loop counter into R0
    LDR R1, =1       ; Load increment value into
R1
    MOV R2, #0       ; Initialize R2 to accumulate

```

the result

; Loop starts

loop_start

ADD R2, R2, R1 ; Add R1 to R2

SUBS R0, R0, #1 ; Decrement R0 and set

flags

BNE loop_start ; Branch back to loop_start

if R0 != 0

; Simulating a system call (e.g., printing R2)

MOV R7, #4 ; System call number for

"print" (Linux syscall convention)

MOV R0, R2 ; Pass the result in R0

SWI 0 ; Software interrupt to invoke

the system call

; Program end

stop

B stop ; Infinite loop to halt the

program

END

Explanation of Key Concepts

1. System Calls:

- SWI 0 simulates a software interrupt to invoke a system-level function like printing.

2. Interrupts:

- The SWI (Software Interrupt) instruction is used here as a demonstration of interrupt handling.

3. Loops:

- A decrementing counter (R0) is used to control the loop's iteration.

4. Branches:

- BNE is used to branch back to the loop start if the condition (R0 != 0) holds.

Expected Output

If running in a simulated ARM environment with system call support, the result of the loop (sum of 10 ones, i.e., 10) will be printed.

Program to demonstrate Time delay program using built in Timer

Aim

To write a program for the LPC2148 microcontroller to demonstrate time delay using Timer0, with an interrupt service routine (ISR) to toggle LEDs periodically.

Procedure

1. Include Header File:

- Include the LPC214x.h header file to access register definitions and macros specific to the LPC2148 microcontroller.

2. Configure GPIO Pins:

- Set the direction of pins P1.16 to P1.23 as output by configuring the IODIR1 register.

3. Configure Timer0:

- Set the T0MR0 register with a match value corresponding to a 1-second delay (15,000,000 counts for a 15 MHz clock).
- Configure the T0MCR register to generate an interrupt and reset the timer when the match value is reached.
- Enable Timer0 by writing 1 to the T0TCR register.

4. Set Up the Interrupt Controller:

- Assign the Timer0 interrupt service routine (ISR) address to the VICVectAddr4 register.
- Configure the interrupt channel in the VICVectCntl4 register.
- Enable the Timer0 interrupt by setting the corresponding bit in the VICIntEnable register.

5. Interrupt Service Routine (ISR):

- In the T0ISR function:
 - Toggle the state of LEDs connected to pins P1.16 to P1.23 using the IOPIN1 register.
 - Clear the interrupt flag in the T0IR register to acknowledge the interrupt.
 - Write to the VICVectAddr register to signal the end of the interrupt.

6. Enter Main Loop:

- Use an infinite loop (while(1)) to keep the microcontroller running indefinitely.

7. Build and Test:

- Compile the program and load it onto the LPC2148 microcontroller.
- Verify that the LEDs toggle at 1-second intervals as expected.

SOURCE CODE:

```

#include <LPC214x.H>
void T0isr(void)    __irq;

void main(void)
{
    IODIR1          =    0xFF << 16; // Configure P1.16 - P1.23 as Output Pins
    T0MR0           = 14999999;        // 1000mSec = 15000.000-1 counts
    T0MCR           = 3;                // Interrupt and Reset on MR0
    T0TCR           = 1;                // Timer0 Enable
    VICVectAddr4 = (unsigned)T0isr;     // Set the timer ISR vector address
    VICVectCntl4 = 0x00000024;          // Set channel
    VICIntEnable |= 0x00000010;         // Enable the interrupt Timer0

    /* ----- 1000 ms Delay Used for Toggling LED ----- */

    while(1);                          //Stay Here...
}

void T0isr (void)    __irq
{
    IOPIN1          =    ~(IOPIN1 & 0x00FF0000); // Toggle LED's for every
    1000ms
    T0IR            |= 0x00000001;                // Clear match 0 interrupt
    VICVectAddr = 0x00000000;                      // Dummy write to signal
    end of interrupt
}

```

Expected Output

- LEDs connected to pins P1.16 to P1.23 toggle every second, demonstrating the use of Timer0 and interrupts for time delay.

Program to Interface 8 Bit LED and Switch Interface

Aim of the Program:

The given program is written in ARM assembly language for an embedded system (likely running on an ARM-based microcontroller). The primary aim of the program is to toggle the state of an I/O pin (presumably connected to an LED or similar output device) in a loop, with a delay between the state changes. This creates a blinking effect for the connected LED or device. Specifically, the program toggles the state of the pins defined by IO1PIN (which controls the I/O pins on port 1 of the microcontroller) and introduces a delay between each toggle to create a visible blinking effect.

Key Operations:

1. **I/O Pin Setup:** The program configures the direction of the I/O pins on port 1 (IO1DIR). The bits corresponding to pins 16–23 are set to output mode (0x00FF0000, which means setting the bits 16 to 23 to 1 for output).
2. **Pin Toggle:**
 - It first loads IO1PIN (which holds the current state of the pins) and writes a value (0x00FF0000) to set the corresponding pins high.
 - A delay is introduced by calling the __delay function.
 - Then it writes 0x00000000 to set the corresponding pins low.
 - Another delay is introduced.
 - This loop repeats continuously.
3. **Delay Function:**
 - The __delay function generates a delay by looping a fixed number of times (3000000 iterations). Each iteration corresponds to a small time delay, and this effectively controls the blinking rate of the LED or device.
 - The delay count is based on the assumption of a 30 MHz clock frequency (each clock cycle is 1/30,000,000 seconds), and the program calculates that
 -

30 cycles correspond to approximately 1 microsecond. Therefore, for a 1 ms delay, 30,000 cycles are needed, which is scaled down to 15,000 due to the two-cycle overhead of the SUBS and BNE instructions.

Procedure:

1. Initialization:

- Set the direction of pins 16–23 on port 1 to output mode using IO1DIR.
- Write 0x00FF0000 to IO1PIN to set the corresponding pins high (turn the LED or output device on).

2. Main Loop:

- The program enters a continuous loop where it:
 - Sets the pins to high (turning on the connected LED or device).
 - Calls the `__delay` function to wait for a certain amount of time.
 - Sets the pins to low (turning off the connected LED or device).
 - Calls the `__delay` function again to wait before turning the LED back on.

3. Delay Function:

- The `__delay` function generates a fixed time delay by counting down from a preset value (3000000), with each iteration corresponding to a small time unit based on the system clock.

4. Repeat:

- The program continuously toggles the state of the I/O pins, creating a blinking effect on the connected device.

SOURCE CODE:

```

GLOBAL __main
        AREA main, CODE, READONLY
        EXPORT __main
        EXPORT __use_two_region_memory
__use_two_region_memory EQU 0
        BX    LR
        ENTRY

```

;Addresses of Registers

```

IO0DIR      EQU      0xE0028008
IO1DIR      EQU      0xE0028018
IO0PIN      EQU      0xE0028000
IO1PIN      EQU      0xE0028010
IO0SET      EQU      0xE0028004
IO0CLR      EQU      0xE002800C
IO1CLR      EQU      0xE002801C
IO1SET      EQU      0xE0028014
PINSEL0     EQU      0xE002C000
PINSEL1     EQU      0xE002C004
PINSEL2     EQU      0xE002C014

```

```

;
__main

        ldr    r1,=IO1DIR          ;IO1DIR
        ldr    r2,=0x00FF0000
        str    r2, [r1]

```

```

loop

        ldr    r1,=IO1PIN          ;IO1PIN
        ldr    r2,=0x00FF0000
        str    r2, [r1]
        BL     __delay

        ldr    r1,=IO1PIN
        ldr    r2,=0x00000000

```



```
str    r2, [r1]
```

```
BL     __delay
```

```
B      loop
```

```
;Delay loop
```

```
;Constant calculations
```

```
;1/30Mhz=1/30us=0.03334us
```

```
;1us requires 30 counts ie 30x0.0334=1us
```

```
;So 1ms requires 30x1000=30000 counts but the SUBS and BNE combination  
requires 2cy
```

```
;So the constant final =30000/2=15000;
```

```
__delay
```

```
STMDB  sp!, {r14}
```

```
LDR     r12, =3000000
```

```
LOOP
```

```
SUBS    r12, r12, #1
```

```
BNE     LOOP
```

```
LDMIA   sp!, {r14}
```

```
BX      r14
```

```
END
```

Result

This program is intended for a microcontroller that controls an output device (e.g., an LED) connected to port 1, pins 16 to 23. The LED blinks on and off with a delay between transitions. The delay is carefully calculated to ensure the blinking happens at a visible rate (approximately every 1 ms).

Program to implement Buzzer Interface

Aim of the Program:

The aim of this program is to control a buzzer (or similar output device) connected to pin 7 of port 0 of the LPC214x microcontroller. The program toggles the state of the buzzer by turning it on and off with a delay between each state change, creating a pulsed sound or signal. The buzzer will continuously blink in a loop.

Key Operations:

1. GPIO Configuration:

- The program first configures port 0, pin 7 (i.e., P0.7) to be used as a General-Purpose Input/Output (GPIO) pin.
- The PINSEL0 register is set to 0x00 to ensure that P0.7 is configured as a GPIO pin (not for any alternate function).
- The IODIR0 register is configured to make P0.7 an output pin by shifting 3 to the left by BUZZ (which is defined as 7), effectively setting bit 7 of IODIR0 to 1 (output).

2. Main Loop:

- Inside the infinite while(1) loop:
 - The program turns the buzzer on by setting P0.7 high using `IOSET0 = 1 << BUZZ;`
 - It waits for a brief period by calling the Delay function.
 - Then, it turns the buzzer off by clearing P0.7 low with `IOCLR0 = 1 << BUZZ;`
 - The program waits again using Delay.
- This loop continues indefinitely, making the buzzer pulse on and off at a regular interval.

3. Delay Function:

- The Delay function generates a simple delay by using nested for loops.
- The outer loop runs 1000 times, and the inner loop runs 700 times for each iteration of the outer loop, resulting in a longer delay. This delay determines how long the buzzer stays on or off before switching states.

The exact time for the delay depends on the clock frequency of the microcontroller. The loop count values (1000 and 700) can be adjusted for longer or shorter delays.

4. Wait Function:

- The Wait function isn't used in the code but might have been intended to serve as an alternative delay function.

Procedure:

1. GPIO Pin Configuration:

- The program configures port 0, pin 7 (P0.7) as a GPIO output pin by writing 0x00 to PINSEL0 to set it as a GPIO and then setting IODIR0 to configure pin 7 as an output.

2. Main Loop:

- The program enters an infinite loop where it:
 - Sets P0.7 high to turn on the buzzer.
 - Calls the Delay function to wait for a set amount of time.
 - Sets P0.7 low to turn off the buzzer.
 - Calls the Delay function again to wait before the next toggle.
- This process continues, creating a pulsed on/off signal for the buzzer.

3. Delay Function:

- The Delay function provides a simple software delay using nested loops. The values in the loops define the duration of the delay, affecting how fast the buzzer pulses on and off.

Result

This program is designed to toggle the state of a buzzer connected to pin 7 of port 0 on the LPC214x microcontroller. The buzzer is turned on and off with a delay in between, creating a pulsed sound or signal. The delay between toggles is managed using a simple nested loop, and the program runs in an infinite loop, continuously pulsing the buzzer.

Program to demonstrate I2C Interface on IDE environment

Aim of the Program:

This program demonstrates basic I2C communication on an LPC214x microcontroller. It shows how to configure the I2C interface, send a start condition, write data to an I2C slave device, and read data from it. It can be used as a foundation for interfacing with various I2C devices like sensors, EEPROMs, or display modules.

Procedure:

1. Initialize the I2C interface and configure the pins for I2C communication.
2. Write data to an I2C device using the I2C_WriteData function.
3. Read data from the I2C device using the I2C_ReadData function.
4. The main loop demonstrates writing data to the I2C slave and reading it back.

Source code:

```
#include <LPC214x.h>
#include <stdio.h>

#define I2C_ADDR 0xA0 // I2C Slave address (for EEPROM or other devices)
#define SCL_PIN 29 // Pin for SCL (I2C Clock)
#define SDA_PIN 30 // Pin for SDA (I2C Data)

// Define I2C Control Register bits
#define I2C_CONSET_I2EN 0x40 // I2C Enable
#define I2C_CONSET_STA 0x20 // Start condition
#define I2C_CONSET_STO 0x10 // Stop condition
#define I2C_CONSET_SI 0x08 // Interrupt flag
#define I2C_CONSET_AA 0x04 // Assert Acknowledge

void I2C_Init(void);
```

```
void I2C_Start(void);
void I2C_Stop(void);
void I2C_Write(unsigned char data);
unsigned char I2C_Read(void);
void I2C_WriteData(unsigned char addr, unsigned char data);
unsigned char I2C_ReadData(unsigned char addr);

int main(void)
{
    I2C_Init(); // Initialize I2C interface

    // Example: Write data to I2C device
    I2C_WriteData(0x01, 0x55); // Write 0x55 to device at address 0x01

    // Example: Read data from I2C device
    unsigned char read_data = I2C_ReadData(0x01);
    printf("Read Data: %x\n", read_data);

    while(1)
    {
        // Main loop
    }

    return 0;
}

void I2C_Init(void)
{
```

```

// Configure the SCL and SDA pins as I2C
PINSEL0 |= (1 << SCL_PIN) | (1 << SDA_PIN); // SCL and SDA pin selection (for
LPC214x)

// Set I2C clock frequency (assuming system clock is 60 MHz, set to 100 kHz for
I2C)
I2C0SCLH = 0x3C; // High byte of I2C clock divider
I2C0SCLL = 0x3C; // Low byte of I2C clock divider

// Enable the I2C interface
I2C0CONSET = I2C_CONSET_I2EN;
}

void I2C_Start(void)
{
    I2C0CONSET = I2C_CONSET_STA; // Generate START condition
    while (!(I2C0CONSET & I2C_CONSET_SI)); // Wait for the interrupt flag (SI)
    I2C0CONCLR = I2C_CONSET_STA; // Clear the START condition
}

void I2C_Stop(void)
{
    I2C0CONSET = I2C_CONSET_STO; // Generate STOP condition
    while (I2C0CONSET & I2C_CONSET_STO); // Wait until STOP condition is
completed
}

void I2C_Write(unsigned char data)
{

```



```
I2C0DAT = data; // Load data to be sent
I2C0CONSET = I2C_CONSET_SI; // Generate interrupt (send data)
while (!(I2C0CONSET & I2C_CONSET_SI)); // Wait until the data is transmitted
I2C0CONCLR = I2C_CONSET_SI; // Clear the SI flag
}

unsigned char I2C_Read(void)
{
    I2C0CONSET = I2C_CONSET_AA; // Assert Acknowledge
    I2C0CONSET = I2C_CONSET_SI; // Generate interrupt (read data)
    while (!(I2C0CONSET & I2C_CONSET_SI)); // Wait for data reception
    I2C0CONCLR = I2C_CONSET_SI; // Clear the SI flag

    return I2C0DAT; // Return received data
}

void I2C_WriteData(unsigned char addr, unsigned char data)
{
    I2C_Start(); // Send Start condition
    I2C_Write(I2C_ADDR); // Send slave address (write mode)
    I2C_Write(addr); // Send register address (memory location or control register)
    I2C_Write(data); // Send data to the register
    I2C_Stop(); // Send Stop condition
}

unsigned char I2C_ReadData(unsigned char addr)
{
    unsigned char data;
```

```

I2C_Start(); // Send Start condition
I2C_Write(I2C_ADDR); // Send slave address (write mode)
I2C_Write(addr); // Send register address (memory location or control register)

I2C_Start(); // Send repeated Start condition
I2C_Write(I2C_ADDR | 0x01); // Send slave address (read mode)
data = I2C_Read(); // Receive data from the register
I2C_Stop(); // Send Stop condition

return data; // Return the received data
}

```

Explanation of the Code:

1. I2C Initialization (I2C_Init):

- The SCL (clock) and SDA (data) pins are configured to work in I2C mode using the PINSEL0 register.
- The I2C interface is initialized by setting the I2C0CONSET = 0x40, which enables the I2C interface.
- The clock divider registers, I2C0SCLH and I2C0SCLL, are set to create a clock frequency of 100 kHz for I2C communication (assuming a 60 MHz system clock).

2. I2C Start and Stop Condition (I2C_Start and I2C_Stop):

- The I2C_Start function generates a START condition by setting the STA bit in the I2C control register.
- The I2C_Stop function generates a STOP condition by setting the STO bit in the control register.

3. I2C Write and Read (I2C_Write and I2C_Read):

- The I2C_Write function writes a byte to the I2C data register and waits for the transmission to complete.
- The I2C_Read function reads a byte from the I2C data register after

asserting the Acknowledge bit (AA).

4. Write and Read Data to/from I2C Device (I2C_WriteData and I2C_ReadData):

- The I2C_WriteData function sends a byte of data to a specific register address in the I2C slave device.
- The I2C_ReadData function reads a byte of data from a specific register address in the I2C slave device by first sending a start condition and writing the register address, followed by a repeated start to read the data.

Program to demonstrate I2C Interface – Serial EEPROM

Aim of the Program:

The aim of this program is to demonstrate basic I2C communication with a serial EEPROM (like the 24C02) on an LPC214x microcontroller. It shows how to write a byte of data to the EEPROM and read a byte of data from it using the I2C interface.

Procedure:

1. Initialize the I2C interface and configure the pins for I2C functionality.
2. Use the I2C_WriteByte function to write a byte of data to a specific address in the EEPROM.
3. Use the I2C_ReadByte function to read a byte of data from the EEPROM and print the result.
4. The program continuously communicates with the EEPROM in a simple manner demonstrating basic I2C operations such as sending start/stop conditions, addressing, reading, and writing.

```
#include <LPC214x.h>
```

```
#include <stdio.h>
```

```
#define EEPROM_ADDR 0xA0 // EEPROM I2C Slave Address for write operation
```

```
#define EEPROM_READ_ADDR 0xA1 // EEPROM I2C Slave Address for read operation
```

```
#define SCL_PIN 29 // Pin for SCL (I2C Clock)
```

```
#define SDA_PIN 30 // Pin for SDA (I2C Data)
```

```
// Define I2C Control Register bits
```

```
#define I2C_CONSET_I2EN 0x40 // I2C Enable
```

```
#define I2C_CONSET_STA 0x20 // Start condition
```

```
#define I2C_CONSET_STO 0x10 // Stop condition
```

```
#define I2C_CONSET_SI 0x08 // Interrupt flag
#define I2C_CONSET_AA 0x04 // Assert Acknowledge

void I2C_Init(void);
void I2C_Start(void);
void I2C_Stop(void);
void I2C_Write(unsigned char data);
unsigned char I2C_Read(void);
void I2C_WriteByte(unsigned char addr, unsigned char data);
unsigned char I2C_ReadByte(unsigned char addr);

int main(void)
{
    // Initialize the I2C interface
    I2C_Init();

    // Example: Write a byte to EEPROM (address 0x10, data 0x55)
    I2C_WriteByte(0x10, 0x55); // Write data to EEPROM address 0x10

    // Example: Read a byte from EEPROM (address 0x10)
    unsigned char read_data = I2C_ReadByte(0x10);
    printf("Read Data: %x\n", read_data);

    while (1)
    {
        // Main loop
    }
}
```

```
    return 0;
}

void I2C_Init(void)
{
    // Configure the SCL and SDA pins for I2C functionality
    PINSEL0 |= (1 << SCL_PIN) | (1 << SDA_PIN); // Set SCL and SDA pins to I2C
mode

    // Set I2C clock frequency (assuming system clock is 60 MHz, set to 100 kHz for
I2C)
    I2C0SCLH = 0x3C; // High byte of I2C clock divider
    I2C0SCLL = 0x3C; // Low byte of I2C clock divider

    // Enable the I2C interface
    I2C0CONSET = I2C_CONSET_I2EN;
}

void I2C_Start(void)
{
    I2C0CONSET = I2C_CONSET_STA; // Generate START condition
    while (!(I2C0CONSET & I2C_CONSET_SI)); // Wait for the interrupt flag (SI)
    I2C0CONCLR = I2C_CONSET_STA; // Clear the START condition
}

void I2C_Stop(void)
{
    I2C0CONSET = I2C_CONSET_STO; // Generate STOP condition
    while (I2C0CONSET & I2C_CONSET_STO); // Wait until STOP condition is
```

```
completed
```

```
}
```

```
void I2C_Write(unsigned char data)
```

```
{
```

```
    I2C0DAT = data; // Load data into the I2C data register
```

```
    I2C0CONSET = I2C_CONSET_SI; // Generate interrupt (send data)
```

```
    while (!(I2C0CONSET & I2C_CONSET_SI)); // Wait until the data is transmitted
```

```
    I2C0CONCLR = I2C_CONSET_SI; // Clear the SI flag
```

```
}
```

```
unsigned char I2C_Read(void)
```

```
{
```

```
I2C0CONSET = I2C_CONSET_AA; // Assert Acknowledge
```

```
    I2C0CONSET = I2C_CONSET_SI; // Generate interrupt (read data)
```

```
    while (!(I2C0CONSET & I2C_CONSET_SI)); // Wait for data reception
```

```
    I2C0CONCLR = I2C_CONSET_SI; // Clear the SI flag
```

```
    return I2C0DAT; // Return the received data
```

```
}
```

```
void I2C_WriteByte(unsigned char addr, unsigned char data)
```

```
{
```

```
    // Send Start condition
```

```
    I2C_Start();
```

```
    // Send the EEPROM address (write mode)
```

```
I2C_Write(EEPROM_ADDR);

// Send the memory address where data will be written (e.g., 0x10)
I2C_Write(addr);

// Send the data byte to be written
I2C_Write(data);

// Send Stop condition
I2C_Stop();
}

unsigned char I2C_ReadByte(unsigned char addr)
{
    unsigned char data;

    // Send Start condition
    I2C_Start();

    // Send the EEPROM address (write mode)
    I2C_Write(EEPROM_ADDR);

    // Send the memory address to read from (e.g., 0x10)
    I2C_Write(addr);

    // Send a repeated Start condition
    I2C_Start();
```



```
// Send the EEPROM address (read mode)
I2C_Write(EEPROM_READ_ADDR);

// Read the data byte
data = I2C_Read();

// Send Stop condition
I2C_Stop();

return data; // Return the read data
}
```

Result

This program demonstrates how to interface with a serial EEPROM over the I2C bus using the LPC214x microcontroller. It provides basic functions to write and read data from the EEPROM, making it a foundation for more complex I2C communication tasks.

Blinking LED

Aim of the Program:

The aim of this program is to demonstrate the use of GPIO pins on the LPC214x microcontroller for controlling LEDs and reading a switch input. The program blinks LEDs connected to Port 1.16 to Port 1.23 and uses a switch connected to Port 1.24 for controlling the LEDs' state. The LEDs will blink in a pattern, and the state of the LEDs will be toggled when the switch is pressed.

Procedure:

1. Pin Configuration:

- The program configures the pins of the microcontroller for input and output functions:
 - Port 1.16 to Port 1.23 (LED) are configured as output pins to control the LEDs.
 - Port 1.24 (Switch) is configured as an input pin to read the state of the switch.

2. LED Blinking:

- The program continuously blinks the LEDs on Port 1.16 to Port 1.23 in a pattern. The LEDs turn on (IOSET1) for a short delay, then turn off (IOCLR1) for a longer delay, creating a blinking effect.

3. Switch Control:

- The program checks the status of the switch on Port 1.24:
 - When the switch is pressed (assuming active-low logic), the status of the LEDs is toggled. If the switch is not pressed, the LEDs blink on and off in the pattern.

4. Infinite Loop:

- The program continuously loops, blinking the LEDs and checking the switch state. The blinking pattern continues until the switch is pressed, at which point the LEDs toggle states.

5. Delay:

- A simple delay function (Delay()) is used to create the timing for the LED blink pattern and switch debounce. The delay is implemented using nested loops that provide a rough approximation of time.

Key Functions:

1. PINSEL2 and IO1DIR Configuration:

- The PINSEL2 register is modified to configure Port 1.16 to Port 1.31 as GPIO (General Purpose I/O) pins.
- The IO1DIR register is used to configure Port 1.16 to Port 1.23 as output pins (for LEDs), and Port 1.24 as an input pin (for the switch).

2. LED Control:

- The LEDs are controlled using the IOSET1 and IOCLR1 registers. IOSET1 turns the LEDs on by setting the corresponding bits in the register, and IOCLR1 turns the LEDs off by clearing those bits.

3. Switch Input:

- The switch state is read using the IO1PIN register. The program checks the status of the switch by masking out the relevant bit and shifting it into the least significant position.

4. Delay:

- The Delay() function introduces a simple delay by running a loop for a certain number of iterations. This function is used to control the timing of LED blinking and switch polling.

Program Flow:

1. The microcontroller initializes the pins for input (switch) and output (LEDs).
2. The program enters an infinite loop:
 - It first blinks the LEDs on and off using IOSET1 and IOCLR1, with delays in between.
 - It then checks the status of the switch using the IO1PIN register. If the switch is pressed (active-low), the LEDs' state is toggled (on/off).
3. The Delay() function is used to control the timing of the LED blinking and the switch polling.

```
#include <LPC214x.h>
#include <stdio.h>
```

```
#define LED 16
#define Switch 24
```

```
/****** Declarations
******/
```

```
void Delay(int);
```

```
/******
******/
```

```
int main(void)
```

```
{
```

```
    unsigned char Status=1;
```

```
    PINSEL2  &=  0xFFFFFFF3;          //Configure P1.16 - P1.31 as GPIO
```

```
    IO1DIR   =   0x00 << Switch; //Configure P1.24 - P1.31 as Input
```

```
    IO1DIR   |=   0xFF << LED;      //Configure P1.16 - P1.23 as Output
```

```
    IO1PIN   =   0;
```

```
    while(1)
```

```
    {
```

```
        Status = 1;
```

```
        IOSET1  =   0xFF << LED;
```

```
        Delay (10);Delay (10);
```

```
        IOCLR1  =   0xFF << LED;
```

```
        Delay (10);Delay (10);Delay (10);
```

```
        while (~Status)
```

```
        {
```

```
            Status      =   ((IO1PIN & (0xFF << Switch)) >> Switch);
```

```
            IO1PIN      =   ((~Status) << LED);
```

```
        }
```

```
    }
```

```
}
```

```
void Delay(int n)
```

```
{
```

```
    int p,q;
```

```
for(p=0;p<n;p++)  
    {  
        for(q=0;q<0x9990;q++);  
    }  
}
```

Result

This program demonstrates basic GPIO handling on an LPC214x microcontroller, where LEDs blink in a pattern and can be toggled based on a switch input. It utilizes the IO1DIR, IO1PIN, IOSET1, and IOCLR1 registers for controlling the pins and reading the input, and a simple delay function to manage timing. The LEDs' state can be toggled by pressing the switch, demonstrating the use of both input and output functionalities.

LCD Display

Aim of the Program:

The aim of this program is to interface and control an LCD display using the LPC214x microcontroller. The program demonstrates how to send commands and data to the LCD to display messages on it. Specifically, the program initializes the LCD and displays two lines of text, ">PS-Primer 2148<" on the first line and ":: LCD Demo! ::" on the second line.

Procedure:

1. Pin Configuration:

- The first step in the program is to configure the pins used for controlling the LCD. The program uses GPIO pins (P0.16 to P0.22) for sending data and control signals (RS, RW, EN) to the LCD.
- These pins are configured as output pins using the IODIR0 register.

2. LCD Initialization:

- The `lcd_initialize()` function is used to initialize the LCD.
- It sends a series of commands to the LCD (stored in the `cmd` array) to initialize it in 4-bit mode and set the display properties (such as cursor position, display on/off, etc.).

3. LCD Command and Data Sending:

- The `lcd_cmd()` function is used to send commands to the LCD (e.g., to set the cursor position, clear the screen, etc.). The RS pin is cleared to indicate that we are sending a command, and the RW pin is also cleared to indicate write operation.
- The `lcd_data()` function is used to send data to the LCD for display. The RS pin is set to indicate data is being sent, and the RW pin is cleared for write operation.
- Both functions use the `LCD4_Convert()` helper function to send the data in 4-bit mode to the LCD.

4. Displaying Messages on LCD:

- The `lcd_display()` function is responsible for displaying the message on the LCD.

- The function first sends the command 0x80 to set the cursor at the beginning of the first line, then it sends the characters from the msg array one by one.
- After the first line is displayed, the function sends the command 0xC0 to move the cursor to the beginning of the second line and then displays the characters from the msg1 array.

5. Delay Function:

- The delay() function is used to introduce a delay between the commands and data sent to the LCD. This ensures that the LCD has enough time to process the commands and display the data correctly.

6. Main Loop:

- The main() function initializes the LCD and calls the lcd_display() function to display the messages on the LCD.
- The program runs indefinitely in the main loop (while(1)).

Key Functions in the Program

1. lcd_cmd(unsigned char data):

- This function sends a command byte to the LCD by clearing the RS and RW pins, and calling LCD4_Convert() to send the data in 4-bit mode.

2. lcd_data(unsigned char data):

- This function sends a data byte to the LCD by setting the RS pin and clearing the RW pin, and calling LCD4_Convert() to send the data in 4-bit mode.

3. lcd_initialize(void):

- This function initializes the LCD by sending a series of initialization commands to the LCD.

4. lcd_display(void):

- This function displays the messages stored in the msg and msg1 arrays on the LCD, with the first message on the first line and the second message on the second line.

5. LCD4_Convert(unsigned char c):

- This function converts the 8-bit data to 4-bit mode and sends it to the LCD using the control pins. It sends the high and low nibbles of the data

byte sequentially.

6. delay(unsigned int n):

- This function introduces a delay by running nested loops to create a time delay. It is used to give the LCD time to process the commands and display the characters.

Program Flow:

1. Pin Setup:

- Pins P0.16 to P0.22 are configured as output pins for controlling the LCD.

2. LCD Initialization:

- The LCD is initialized by sending a series of commands to the LCD to configure it in 4-bit mode, enable the display, and set the cursor properties.

3. Display Messages:

- Two messages are displayed on the LCD using the lcd_display() function. The first message appears on the first line, and the second message appears on the second line.

4. Endless Loop:

- After displaying the messages, the program enters an infinite loop (while(1)), keeping the messages displayed on the LCD.

```
#include <LPC214x.H>
#define RS0x10000
#define RW    0x20000
#define EN    0x40000
void lcd_cmd (unsigned char);
void lcd_data (unsigned char);
void lcd_initialize (void);
void lcd_display (void);
void LCD4_Convert(unsigned char);
const unsigned char cmd[4] = {0x28,0x0c,0x06,0x01}; //lcd commands
unsigned char msg[] = {">PS-Primer 2148<"}; //msg
unsigned char msg1[] = {":: LCD Demo! ::"}; //msg1
void delay(unsigned int n)
{
```



```

    unsigned int i,j;
    for(i=0;i<n;i++)
    {
        for(j=0;j<12000;j++)
        {;}
    }
}
//-----
//          LCD Command Send
//-----
void lcd_cmd(unsigned char data)
{

    IOCLR0  |= RS;          //0x1000;      //RS
    IOCLR0  |= RW;          //0x2000;      //RW
    LCD4_Convert(data);
}
void lcd_initialize(void)
{
    int i;
    for(i=0;i<4;i++)
    {
        IOCLR0 = 0xF << 19;          //IOCLR 0/1
        lcd_cmd(cmd[i]);
    }
    delay(15);
}
//-----
//          LCD Data Send
//-----
void lcd_data (unsigned char data)
{

    IOSET0   |= RS;          //0x1000;      //RS
    IOCLR0   |= RW;          //0x2000;      //RW
    LCD4_Convert(data);
}
//

```

```

//          LCD Display Msg
//-----
void lcd_display (void)
{
    char i;

    /* first line message */
    lcd_cmd(0x80);
    delay(15);
    i=0;
    while(msg[i]!='\0')
    {
        delay(5);
        lcd_data(msg[i]);
        i++;
    }
    delay(15);
    /* second line message */
    lcd_cmd(0xc0);
    delay(15);
    i=0;
    while(msg1[i]!='\0')
    {
        delay(5);
        lcd_data(msg1[i]);
        i++;
    }
    delay(15);
}

void LCD4_Convert(unsigned char c)
{
    if(c & 0x80) IOSET0 = 1 << 22; else IOCLR0 = 1 << 22;
    if(c & 0x40) IOSET0 = 1 << 21; else IOCLR0 = 1 << 21;
    if(c & 0x20) IOSET0 = 1 << 20; else IOCLR0 = 1 << 20;
    if(c & 0x10) IOSET0 = 1 << 19; else IOCLR0 = 1 << 19;
}

```

```

IOSET0  = EN;
//0x4000;      //EN
delay(15);
IOCLR0  = EN;
//0x4000;      //EN

if(c & 0x08) IOSET0 = 1 << 22; else IOCLR0 = 1 << 22;
if(c & 0x04) IOSET0 = 1 << 21; else IOCLR0 = 1 << 21;
if(c & 0x02) IOSET0 = 1 << 20; else IOCLR0 = 1 << 20;
if(c & 0x01) IOSET0 = 1 << 19; else IOCLR0 = 1 << 19;

IOSET0  = EN;
//0x4000;      //EN
delay(15);
IOCLR0  = EN;
//0x4000;      //EN
}
void main()
{
    PINSEL1 = 0;                                     //Configure P0.16
P0.31 as GPIO
    IODIR0  = 0xFF << 16;    //Configure Pins P0.16 - P0.22 as Output Pins
    lcd_initialize();         //Initialize LCD!
    lcd_display();            //Display Message in LCD
    while(1);
}

```

Result

This program demonstrates how to interface an LCD with the LPC214x microcontroller using 4-bit data mode. The program initializes the LCD, sends commands and data to control the display, and displays two messages on the LCD. The program also uses a delay function to ensure proper timing for the LCD to process commands and display data.